claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_d9866476-800\agent-af8dcb308e52d89d4.jsonl`
- SHA-256: `3c3662b5c52c83532dbc7095fb8ab3fe820bfc4327ebcc037db0000d9e5e1805`
- Source modified: `2026-06-22T09:10:18+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d9866476-800`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:08:03.300Z)

You are mapping the badminton-highlight-indexer repo, checked out at origin/master under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw structured data, not prose for a human.

MAP THE REEL-PRODUCTION PATH end-to-end and define precisely what "number of reels" means as a countable, assertable integer. Trace: backend/main.py (::process_video /api/process, ::compile_highlights /api/compile, ::_execute_processing), the stitcher/stitching module, config stitching (min_shot_count, padding), how detected segments/rallies become highlight clips, and whether the output is ONE stitched reel per video or N separate highlight clips. Identify the EXACT integer that is the stable regression signal (e.g. count of play_segments rows in the DB, count of stitched output files, count of compiled highlights) and how to read it headlessly (CLI flag / API / DB query). Give the exact headless command(s) to process a video + produce reels without the web UI.

2. assistant (2026-06-22T09:08:05.640Z)

I'll map the reel-production path end-to-end in the badminton-highlight-indexer repo to define the regression test signal.

3. user (2026-06-22T09:08:06.739Z)

.claude\worktrees\nightly-e2e\backend\annotations__init__.py
.claude\worktrees\nightly-e2e\backend\annotations\automated_provider.py
.claude\worktrees\nightly-e2e\backend\annotations\base.py
.claude\worktrees\nightly-e2e\backend\annotations\file_provider.py
.claude\worktrees\nightly-e2e\backend\api__init__.py
.claude\worktrees\nightly-e2e\backend\api\auth.py
.claude\worktrees\nightly-e2e\backend\api\job_worker.py
.claude\worktrees\nightly-e2e\backend\api\uploads.py
.claude\worktrees\nightly-e2e\backend\billing.py
.claude\worktrees\nightly-e2e\backend\compute__init__.py
.claude\worktrees\nightly-e2e\backend\compute\base.py
.claude\worktrees\nightly-e2e\backend\compute\cloud_run.py
.claude\worktrees\nightly-e2e\backend\config__init__.py
.claude\worktrees\nightly-e2e\backend\config\loader.py
.claude\worktrees\nightly-e2e\backend\config\models.py
.claude\worktrees\nightly-e2e\backend\config\presets.py
.claude\worktrees\nightly-e2e\backend\config\startup.py
.claude\worktrees\nightly-e2e\backend\eval__init__.py
.claude\worktrees\nightly-e2e\backend\eval\ablation.py
.claude\worktrees\nightly-e2e\backend\eval\ablation_pdf.py

.claude\worktrees\nightly-e2e\backend\eval\annotations.py
.claude\worktrees\nightly-e2e\backend\eval\audio__init__.py
.claude\worktrees\nightly-e2e\backend\eval\audio\e1_probe.py
.claude\worktrees\nightly-e2e\backend\eval\batch.py
.claude\worktrees\nightly-e2e\backend\eval\calibrate_local.py
.claude\worktrees\nightly-e2e\backend\eval\calibrate_wasb.py
.claude\worktrees\nightly-e2e\backend\eval\calibration.py
.claude\worktrees\nightly-e2e\backend\eval\classifier.py
.claude\worktrees\nightly-e2e\backend\eval\distill_local.py
.claude\worktrees\nightly-e2e\backend\eval\eval_partial_labels.py
.claude\worktrees\nightly-e2e\backend\eval\eval_stats.py
.claude\worktrees\nightly-e2e\backend\eval\experiment.py
.claude\worktrees\nightly-e2e\backend\eval\export_wasb_segments.py
.claude\worktrees\nightly-e2e\backend\eval\fusion_audio.py
.claude\worktrees\nightly-e2e\backend\eval\fusion_compare.py
.claude\worktrees\nightly-e2e\backend\eval\fusion_golden.py
.claude\worktrees\nightly-e2e\backend\eval\gemini_refine.py
.claude\worktrees\nightly-e2e\backend\eval\golden_fixtures.py
.claude\worktrees\nightly-e2e\backend\eval\golden_real_fixtures.py
.claude\worktrees\nightly-e2e\backend\eval\gt_loader.py
.claude\worktrees\nightly-e2e\backend\eval\harness.py
.claude\worktrees\nightly-e2e\backend\eval\metrics.py
.claude\worktrees\nightly-e2e\backend\eval\per_user_eval.py
.claude\worktrees\nightly-e2e\backend\eval\per_user_gate.py
.claude\worktrees\nightly-e2e\backend\eval\per_user_regression.py
.claude\worktrees\nightly-e2e\backend\eval\promotion.py
.claude\worktrees\nightly-e2e\backend\eval\rally_seg_eval.py
.claude\worktrees\nightly-e2e\backend\eval\rally_seq_proto.py
.claude\worktrees\nightly-e2e\backend\eval\regression.py
.claude\worktrees\nightly-e2e\backend\eval\score_calibration.py
.claude\worktrees\nightly-e2e\backend\eval\segmentation_metrics.py
.claude\worktrees\nightly-e2e\backend\eval\served_gate_a.py
.claude\worktrees\nightly-e2e\backend\eval\serve_contrast.py
.claude\worktrees\nightly-e2e\backend\eval\splits.py
.claude\worktrees\nightly-e2e\backend\eval\tolerance_metrics.py
.claude\worktrees\nightly-e2e\backend\eval\training.py
.claude\worktrees\nightly-e2e\backend\eval\windowing.py
.claude\worktrees\nightly-e2e\backend\features__init__.py
.claude\worktrees\nightly-e2e\backend\features\rally_seq.py
.claude\worktrees\nightly-e2e\backend\infrastructure\database.py
.claude\worktrees\nightly-e2e\backend\infrastructure\execution_policy.py
.claude\worktrees\nightly-e2e\backend\main.py
.claude\worktrees\nightly-e2e\backend\pipeline\audio__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\audio\hit_detector.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\audio_features.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\court_gate.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\device.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\native_wasb_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\person_detector.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\stub_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\wasb_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\personalization__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters_keyhint.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py

```
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\watermark_i18n.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\base.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\blur.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\format.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\pts_continuity.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\relevance.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\resolution_fps.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\scene_cut.py
.claude\worktrees\nightly-e2e\backend\providers\base.py
(Results are truncated. Consider using a more specific path or pattern.)
```

4. user (2026-06-22T09:08:07.590Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/annotations/automated_provider.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/annotations/base.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/annotations/file_provider.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/annotations/__init__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/api/auth.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/api/job_worker.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/api/uploads.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/api/__init__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/billing.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/compute/base.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/compute/cloud_run.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/compute/__init__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/loader.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/models.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/presets.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/startup.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/__init__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/ablation.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/ablation_pdf.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/annotations.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/audio/e1_probe.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/audio/__init__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/batch.py
```

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/calibrate_local.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/calibrate_wasb.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/calibration.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/classifier.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/distill_local.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/eval_partial_labels.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/eval_stats.py
```

5. assistant (2026-06-22T09:08:09.446Z)

Now let me read the main.py entry point and trace the API endpoints:

6. user (2026-06-22T09:08:11.203Z)

```
1       import os
2       import sys
3       import argparse
4       import json
5       import logging
6       import shutil
7       import socket
8       import threading
9       import uuid
10      import webbrowser
11      from importlib.metadata import PackageNotFoundError, version as _pkg_version
12      from urllib.parse import quote
13      import uvicorn
14      from dotenv import load_dotenv
15
16      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
17      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
18      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
19      # stdlib-only import ? keeps this pre-pydantic line light.
20      from backend.utils.app_home import env_file_to_load, resolve_output_dir
21
22      load_dotenv(env_file_to_load())
23
24      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
25      logging.basicConfig(
26          level=logging.INFO,
27          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
28          datefmt='%H:%M:%S'
29      )
30      logger = logging.getLogger(__name__)
31      from fastapi import FastAPI, HTTPException, Request
32      from fastapi.middleware.cors import CORSMiddleware
33      from starlette.middleware.trustedhost import TrustedHostMiddleware
34      from pydantic import BaseModel
35      from typing import List, Dict, Any, Optional
36
37      from fastapi.staticfiles import StaticFiles
38      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
39      from backend.pipeline.validators.base import registry
40      from backend.infrastructure.database import Database
```

```
41      from backend.pipeline.segmenters.base import segmenter_registry
42      from backend.pipeline.stitcher.compiler import VideoCompiler
43      from backend.pipeline.stitcher.watermark_i18n import localize_watermark
44      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
45      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
46      from backend.utils.redact import redact_secrets  # P0b: mask secret-shaped fields in
/api/config
47      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
48      from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin  # P2a: single ffmpeg/ffprobe
resolver
49      from backend.config import (  # new typed config
50          load_config as load_app_config,
51          write_light_default_config,  # P2b: batteries-included first-run config seeding
52          AppConfig,
53          StitchingConfig,
54          enforce_startup_checks,
55          StartupCheckError,
56      )
57      from backend.results import Failure  # standardized error/result contract
58      from backend.reporting import emit_indexer_report
59      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
60      from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
61      from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-
OFF)
62
63      app = FastAPI(title="Sports Highlight Indexer API")
64
65
66      def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
67          """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
68          default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
69          does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
70          src = os.environ if env is None else env
71          raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
72          origins = [o.strip() for o in raw.split(",") if o.strip()]
73          return origins or ["*"]
74
75
76      # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
77      # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
78      # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
79      # lock it to its origin; the middleware is fixed at startup.
80      app.add_middleware(
81          CORSMiddleware,
82          allow_origins=_cors_origins_from_env(),
83          allow_credentials=False,
84          allow_methods=["*"],
85          allow_headers=["*"],
86      )
87
88
89      def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
90          """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
91          web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
92          the browser sends ``Host: attacker.com``, which this rejects (the server only
```

```
answers loopback
93          Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
94          Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
95          ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
96          deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
97          src = os.environ if env is None else env
98          raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
99          hosts = [h.strip() for h in raw.split(",") if h.strip()]
100         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
101         # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"),
so IPv6
102         # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
103         return hosts or ["localhost", "127.0.0.1", "testserver"]
104
105
106     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
107     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
108     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
109     app.add_middleware(
110         TrustedHostMiddleware,
111         allowed_hosts=_allowed_hosts_from_env(),
112         www_redirect=False,
113     )
114
115
116     def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
117         """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
118         when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
119         in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
120         interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
121         if cli_host and cli_host.strip():
122             return cli_host.strip()
123         env = os.environ.get("RALLY_BIND_HOST")
124         if env and env.strip():
125             return env.strip()
126         sec = getattr(config, "security", None)
127         if getattr(sec, "edge_auth_enabled", False):
128             return "0.0.0.0"  # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
129         return "127.0.0.1"
130
131
132     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
O1) ---
133
134     def _port_is_free(host: str, port: int) -> bool:
135         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
136         uvicorn surfaces a real bind failure loudly)."""
137         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
138             try:
139                 s.bind((host, port))
```

```
140                 return True
141             except OSError:
142                 return False
143 
144 
145     def _find_free_port(host: str) -> int:
146         """An OS-assigned free port on host."""
147         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
148             s.bind((host, 0))
149             return int(s.getsockname()[1])
150 
151 
152     def _resolve_app_port(host: str, port: int) -> int:
153         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
154         return port if _port_is_free(host, port) else _find_free_port(host)
155 
156 
157     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
158         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
159         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
160         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
161         if os.environ.get("RALLY_NO_BROWSER"):
162             return False
163         import time as _time
164         import urllib.request
165         deadline = _time.monotonic() + timeout
166         while _time.monotonic() < deadline:
167             try:
168                 with urllib.request.urlopen(health_url, timeout=2) as r:
169                     if getattr(r, "status", 200) == 200:
170                         break
171             except Exception:
172                 _time.sleep(0.3)
173         else:
174             return False
175         try:
176             webbrowser.open(url)
177         except Exception:
178             pass
179         return True
180 
181 
182     # Serves static frontend files directly (now under api/ per approved structure).
183     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
184     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
185     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
186     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
187     try:
188         os.makedirs(_STATIC_DIR, exist_ok=True)
189     except OSError:
190         # The dir ships with the package (a no-op create); guard against a read-only install
191         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
192         pass
193     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
194 
195     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
196     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
```

```
if it's
197     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
198     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
199     # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
200     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
201     _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
else _STATIC_DIR
202     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
203     if os.path.isdir(_UI_ASSETS_DIR):
204         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
205
206
207     @app.get("/", response_class=HTMLResponse)
208     def serve_index():
209         index_path = os.path.join(_UI_DIR, "index.html")
210         if os.path.exists(index_path):
211             with open(index_path, "r", encoding="utf-8") as f:
212                 return f.read()
213         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
214
215
216     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
217         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
218         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
219         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
220         try:
221             with open(path, "r", encoding="utf-8") as f:
222                 return json.load(f)
223         except (OSError, ValueError):
224             return None
225
226     # Global variables initialized at startup
227     db_instance: Optional[Database] = None
228     global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
229     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
230     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
231     _instance_lock: Optional[Any] = None
232
233     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
234     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
235     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
236     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
237     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
238     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
239     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
240     from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
241         JobWaiting,
242         _arm_wake_timer,
243         _drain_due_jobs,
244         _get_executor,
245         _job_to_request,
246         _MAX_DRAIN_WAKE_SEC,
247         _maybe_record_job_cost,
248         _run_claimed_job,
249         _schedule_next_drain_if_waiting,
```

```
    )

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None
        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
        # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
        locale: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str
        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
        # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
        locale: Optional[str] = None

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
        # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
        # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
        # by tests/test_redact.py + the /api/config redaction test).
        if global_config is None:
            return {}
        return redact_secrets(global_config.model_dump(mode="json"))
```

```
306
307     @app.get("/api/health")
308     def health():
309         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
310         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
311         confirm the engine is reachable and which detector is wired. Never raises."""
312         sport = getattr(global_config, "sport", None)
313         indexing = getattr(global_config, "indexing", None)
314         return {
315             "status": "ok",
316             "sport": sport,
317             "default_segmenter": getattr(indexing, "default_segmenter", None),
318         }
319
320
321     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
322     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
323     API_VERSION = 1
324
325
326     def _engine_version() -> str:
327         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
328         try:
329             return _pkg_version("badminton-highlight-indexer")
330         except PackageNotFoundError:
331             return "unknown"
332
333
334     @app.get("/api/version")
335     def api_version():
336         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
337         return {"engine_version": _engine_version(), "api_version": API_VERSION, "ui":
_bundled_ui_version()}
338
339
340     @app.get("/api/capabilities")
341     def capabilities():
342         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
343         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
344         value. Best-effort + never raises (the wizard degrades gracefully)."""
345         indexing = getattr(global_config, "indexing", None)
346         deployment = getattr(global_config, "deployment", None)
347         ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
348         ffprobe = shutil.which(ffprobe_bin())
349         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
350         wasb = getattr(indexing, "wasb", None)
351         weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
or ""
352         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
353         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
354         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
355         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
356         nvidia_smi = shutil.which("nvidia-smi")
357         return {
358             "api_version": API_VERSION,
```

```
            "engine_version": _engine_version(),
            "segmenters": sorted(segmenter_registry.list_available().keys()),
            "default_segmenter": getattr(indexing, "default_segmenter", None),
            "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
                       "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
            "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
            "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
            "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
            "deployment": {
                "profile": getattr(deployment, "profile", "") or "",
                "compute_target": getattr(deployment, "compute_target", "") or "",
            },
        }


    @app.get("/api/videos")
    def list_videos(limit: Optional[int] = None):
        """List indexed videos (most recent first) so the console survives a reload ? previously only a
        single-row get existed (PACKAGING_PLAN.md P1)."""
        return {"videos": db_instance.list_videos(limit=limit)}


    def _reels_dir() -> str:
        return getattr(getattr(global_config, "output", None), "output_dir", "output") or "output"


    _REEL_EXTS = (".mp4", ".mov")


    @app.get("/api/reels")
    def list_reels():
        """List compiled highlight reels present in the output dir, newest first, with a download URL.

        NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the engine
        does NOT track a video_id?reel association today (a per-video listing would be a phantom), so
        this is a global list. (Per-video reel tracking would need a DB record + compile change ? a
        later slice.) Only top-level video files are listed (per-video scratch subdirs are skipped)."""
        base = _reels_dir()
        reels = []
        try:
            entries = os.scandir(base)
        except OSError:
            return {"reels": []}
        with entries:
            for e in entries:
                if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
                    try:
                        st = e.stat()
                    except OSError:
                        continue
                    reels.append({
                        "filename": e.name,
                        "size_bytes": st.st_size,
                        "modified_at": st.st_mtime,
                        "download_url": f"/api/reels/{quote(e.name)}",
                    })
        reels.sort(key=lambda r: r["modified_at"], reverse=True)
        return {"reels": reels}
```

```
417
418
419     @app.get("/api/reels/{filename}")
420     def download_reel(filename: str):
421         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
422         Mirrors /api/highlights' safety (bare-name + containment checks)."""
423         try:
424             name = safe_output_filename(filename)
425         except ValueError as e:
426             raise HTTPException(status_code=400, detail=str(e)) from e
427         base = _reels_dir()
428         path = os.path.join(base, name)
429         try:
430             path = resolve_under_root(path, base)
431         except ValueError as e:
432             raise HTTPException(status_code=400, detail=str(e)) from e
433         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
434             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
435         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
436         return FileResponse(path, media_type=media, filename=name)
437
438
439     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
440         """The full hash ? validate ? segment ? report pipeline for one video.
441
442         This is the former synchronous /api/process body, unchanged in behavior, now run on
443         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
444         the sync endpoint used to return (UI compatibility); the per-video observability
445         report is still emitted in `finally:` and grafted as response["reports"].
446
447         `progress` is an optional callable(stage: str) used to surface coarse job progress.
448         Raises on unexpected internal errors ? the caller records those as a job failure
449         (after this function has already restored the pre-run segments, see below).
450         """
451         notify = progress or (lambda stage: None)
452
453         notify("hashing")
454         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
455         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
456         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
457         # consumers (hash / validators / segmenter) use the resolved local path.
458         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
459         video_id = compute_video_id(local_video)
460         was_reingest = db_instance.get_video(video_id) is not None
461
462         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
463         notify("validating")
464         logger.info(f"Running validations for {req.video_path}...")
465         val_results, val_context = registry.run_all_with_context(local_video, global_config)
466         failures = [r for r in val_results if not r.passed]
467
468         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
469         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
470         seg_name = req.segmenter or default_seg
471         segmenter_actual = None
472         segmentation_ran = False
473         response: Optional[Dict[str, Any]] = None
474         try:
```

```
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
```

```
532                 return response
533
534             segments = result.value if hasattr(result, "value") else result
535             notify("finalizing")
536             _finalize_reingest(video_id, segments, play_wm, cand_wm)
537
538             response = {
539                 "video_id": video_id,
540                 "status": "success",
541                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
542                 "results": [r.model_dump() for r in val_results],
543                 "play_segments": segments,
544             }
545             return response
546         finally:
547             # Always emit the per-video observability report (next to the video, or to
548             # report.output_dir). Never raises. Surface the paths in the response.
549             paths = emit_indexer_report(
550                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
551                 val_results=val_results, val_context=val_context,
552                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
553                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
554             )
555             if paths and isinstance(response, dict):
556                 response["reports"] = paths
557
558
559     @app.post("/api/process", status_code=202)
560     def process_video(req: ProcessRequest, request: Request):
561         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
562
563         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
564         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
565         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
566         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
567         """
568         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
569         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
570         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
571         try:
572             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
573         except edge_auth.EdgeAuthError as e:
574             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
575
576         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
577         try:
578             validate_input_path(req.video_path, allowed_root=allowed_root)
579             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
580             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
581             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
582         except ValueError as e:
583             raise HTTPException(status_code=400, detail=str(e)) from e
584         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
585         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
586         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
```

```
when the worker
587         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
588         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
589         # rejects a non-local URI as out-of-root.
590         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
591             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
592         if req.segmenter and not segmenter_registry.get(req.segmenter):
593             available = list(segmenter_registry.list_available().keys())
594             raise HTTPException(
595                 status_code=400,
596                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
597             )
598
599         job_id = uuid.uuid4().hex
600         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
601                                       req.player_pool, req.max_frames, owner_id=owner_id,
602                                       locale=req.locale):
603             raise HTTPException(status_code=500, detail="Failed to persist job record.")
604         _get_executor().submit(_drain_due_jobs)
605         return JSONResponse(
606             status_code=202,
607             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
608         )
609
610
611     @app.get("/api/jobs/{job_id}/cost")
612     def get_job_cost(job_id: str):
613         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
614         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
615         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
616         cost = db_instance.get_job_cost(job_id)
617         if cost is None:
618             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
619         fx = global_config.billing.fx_inr_per_usd
620         usd = cost.get("cost_usd")
621         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
622         cost["fx_inr_per_usd"] = fx
623         return cost
624
625
626     @app.get("/api/videos/{video_id}/cost")
627     def get_video_cost(video_id: str):
628         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
629         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
630         agg = db_instance.get_video_cost(video_id)
631         fx = global_config.billing.fx_inr_per_usd
632         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
633         agg["fx_inr_per_usd"] = fx
634         return agg
635
636
637     @app.get("/api/jobs/{job_id}")
638     def get_job(job_id: str):
639         """Job state: {status, progress, result, reports}. `status` = execution state
640         (queued|running|done|failed); the pipeline verdict is result["status"]."""
641         job = db_instance.get_job(job_id)
642         if not job:
```

```
643             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
644         result = job.get("result")
645         return {
646             "job_id": job["id"],
647             "status": job["status"],
648             "progress": job.get("progress"),
649             "video_id": job.get("video_id"),
650             "error": job.get("error"),
651             "result": result,
652             "reports": (result or {}).get("reports"),
653             "created_at": job.get("created_at"),
654             "started_at": job.get("started_at"),
655             "finished_at": job.get("finished_at"),
656         }
657 
658     # --- Chunked upload (B2, PR-2) ----------------------------------------------------
659     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
660     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
661     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
662     # sequential-part logic, and abort cleanup ? is unchanged.
663     from backend.api import uploads as uploads_api
664 
665     app.include_router(uploads_api.router)
666 
667 
668     # --- Highlight download (B3, PR-2) ------------------------------------------------
669 
670     @app.get("/api/highlights/{video_id}/{filename}")
671     def download_highlight(video_id: str, filename: str):
672         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
673         (which only writes into output_dir; the browser previously got back an unreachable
674         server-local path)."""
675         try:
676             name = safe_output_filename(filename)
677         except ValueError as e:
678             raise HTTPException(status_code=400, detail=str(e)) from e
679         if not db_instance.get_video(video_id):
680             raise HTTPException(status_code=404, detail="Indexed video record not found.")
681         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
682         path = os.path.join(output_dir, name)
683         try:
684             path = resolve_under_root(path, output_dir)
685         except ValueError as e:
686             raise HTTPException(status_code=400, detail=str(e)) from e
687         if not os.path.isfile(path):
688             raise HTTPException(status_code=404,
689                                 detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
690         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
691         return FileResponse(path, media_type=media, filename=name)
692 
693 
694     @app.post("/api/query")
695     def query_play_segments(req: QueryRequest):
696         filters = {
697             "server": req.server,
698             "receiver": req.receiver,
699             "duration_min": req.duration_min,
700             "event_type": req.event_type
701         }
702         segments = db_instance.query_play_segments(req.video_id, filters)
```

```
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "
f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
        # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
        # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        localized_watermark = localize_watermark(stitching.watermark, req.locale)
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
```

```
watermark=localized_watermark)
758         try:
759             # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
760             # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
761             # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
762             # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
763             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
764             local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
765             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
766             # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
767             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
768             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
769             return {"status": "success", "filepath": out_path, "download_url": download_url}
770         except Exception as e:
771             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
772
773     # --- CLI Debug Utility & Orchestration ---
774     def run_cli_diagnose_clip(video_path: str, timestamp: float):
775         """CLI debugging utility to examine segment properties around a timestamp."""
776         print("=" * 60)
777         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
778         print("=" * 60)
779
780         cfg = load_config()  # legacy wrapper still works, returns dict for now
781
782         # 1. Validation check diagnostics
783         print("[DIAGNOSTIC] Running validation framework...")
784         results = registry.run_all(video_path, cfg)
785         for r in results:
786             status_symbol = "[PASS]" if r.passed else "[FAIL]"
787             print(f"  {status_symbol} {r.validator_name}: {r.message}")
788             if r.details:
789                 print(f"         Details: {r.details}")
790
791         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
792         print("-" * 60)
793         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
794         import cv2
795         cap = cv2.VideoCapture(video_path)
796         if not cap.isOpened():
797             print("[FAIL] OpenCV could not open video.")
798             return
799
800         fps = cap.get(cv2.CAP_PROP_FPS)
801         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
802
803         start_frame = max(0, int((timestamp - 3.0) * fps))
804         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
805
806         # Measure frame-by-frame changes in sample region
807         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
808         prev_gray = None
809         motions = []
810
811         for _ in range(start_frame, end_frame):
812             ret, frame = cap.read()
```

```
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)


    def _enforce_startup_or_exit(config) -> None:
        """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
        any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
        the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
        auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
        ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
        the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
        fail-fast wiring + the default-OFF no-op are unit-testable."""
        try:
            enforce_startup_checks(config, logger=logger, include_exposure=True)
        except StartupCheckError as e:
            logger.error("[STARTUP] refusing to start: %s", e)
            sys.exit(1)


    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
```

```
865         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
866         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
867         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
868         parser.add_argument("--app", action="store_true",
869                             help="Launch the app: start the local server (localhost-only)
and open the "
870                                  "bundled UI in your browser. Falls back to a free port if
--port is busy.")
871         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
872         parser.add_argument("--host", default=None,
873                             help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
874                                  "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
875         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
876
877         available_segmenters = list(segmenter_registry.list_available().keys())
878         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
879         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
880         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
881         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
882
883         args = parser.parse_args()
884
885         if args.setup:
886             # Invoke the diagnostics/bootstrap script (renamed from the old root
887             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
888             # build system). Resolve its path from this file so `indexer --setup`
889             # works regardless of the caller's cwd.
890             import subprocess
891             doctor_path = os.path.join(
892                 os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
893                 "scripts", "doctor.py",
894             )
895             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
896             subprocess.run([sys.executable, doctor_path])
897             return
898
899         if args.trim_start is not None and args.trim_end is not None:
900             if not args.video_path:
901                 print("[ERROR] Please provide --video-path to extract a segment.")
902                 return
903
904             # Determine output path
905             out_filename = args.trim_output or "trimmed_segment.mp4"
906             if os.path.isabs(out_filename):
907                 out_path = out_filename
908                 out_dir = os.path.dirname(out_path)
909                 out_name = os.path.basename(out_path)
910             else:
911                 out_dir = resolve_output_dir(args.output_dir)  # P0a: app-home aware (CWD-
identical when unset)
912                 out_path = os.path.join(out_dir, out_filename)
913                 out_name = out_filename
914
915             os.makedirs(out_dir, exist_ok=True)
```

```
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            # global_config isn't initialized this early; load the typed config so the trim
            # clip carries the same configured stitching paddings + watermark as a real
            # highlight (watermark default-on).
            stitching = load_app_config().stitching
            compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly
`--app`
        # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
        # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
        # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
        # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
        # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
        # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
        if args.app:
            seeded_path, seeded = write_light_default_config()
            if seeded:
                print(f"[APP] First run: seeded a truly-local default config at
{seeded_path} "
                      f"(detector 'motion' ? offline, no API key or GPU needed). "
                      f"Use the in-app setup to switch to Gemini.")

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig
        _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE
profile

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
        # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
        global_config.output.output_dir = resolve_output_dir(args.output_dir)
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)

```

```
967         # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
968         # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
969         # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
970         # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
971         global _instance_lock
972         try:
973             _instance_lock = acquire_lock(db_path + ".lock")
974         except OSError as exc:
975             # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
976             # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
977             # legitimate single instance).
978             logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
979                            "WITHOUT the second-instance guard.", db_path, exc)
980             _instance_lock = None
981         else:
982             if _instance_lock is None:
983                 print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
984                       f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
985                       f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
986                 sys.exit(1)
987
988         db_instance = Database(db_path)
989
990         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
991         # the executor is in-process. Mark them failed so pollers see a terminal state.
992         swept = db_instance.fail_stale_jobs()
993         if swept:
994             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
995
996         # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
997         if args.video_path and not args.server and not args.app:
998             print(f"[CLI] Processing video file: {args.video_path}")
999             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1000             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1001             # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1002             storage_cfg = getattr(global_config, "storage", None)
1003             try:
1004                 input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1005             except ValueError as e:
1006                 print(f"[FAIL] {e}")
1007                 return
1008             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1009                 print(f"[FAIL] Video file not found: {args.video_path}")
1010                 return
1011             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1012
1013             video_id = compute_video_id(local_video)
1014             was_reingest = db_instance.get_video(video_id) is not None
1015
1016             # Validate (with-context variant surfaces ffprobe_meta for the report)
```

```
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(local_video,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])

            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "="*40)
                print("VALIDATION SUMMARY")
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
                segmentation_ran = True

                if isinstance(result, Failure):
                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                    print("[FAIL] Indexing halted for video.")
                    segments = []
                else:
                    segments = result.value if hasattr(result, "value") else result
                    print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
                # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
                _finalize_reingest(video_id, segments, play_wm, cand_wm)
            finally:
                # Always emit the per-video observability report. Never raises.
                paths = emit_indexer_report(
                    db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
                    val_results=val_results, val_context=val_context,
                    segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                    was_reingest=was_reingest, segmentation_ran=segmentation_ran,
                )
                if paths:
                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
```

```
1075            return
1076
1077        # Start Local Server Mode
1078        if args.app or args.server or not args.video_path:
1079            if args.app:
1080                # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1081                # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1082                bind_host = "127.0.0.1"
1083                port = _resolve_app_port(bind_host, args.port)
1084                if port != args.port:
1085                    print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1086                url = f"http://{bind_host}:{port}/"
1087                # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1088                # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1089                # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1090                print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
1091                      f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1092                threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1093                                 daemon=True).start()
1094            else:
1095                bind_host = _resolve_bind_host(global_config, args.host)  # P0c:
config/profile-driven (default 127.0.0.1)
1096                port = args.port
1097                print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1098                if bind_host not in ("127.0.0.1", "localhost", "::1"):
1099                    print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1100                          f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1101                          f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1102                          f"loopback-only and will 400 forwarded requests otherwise).")
1103            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1104            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1105            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1106            _get_executor().submit(_drain_due_jobs)
1107            uvicorn.run(app, host=bind_host, port=port)
1108
1109    if __name__ == "__main__":
1110        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1111        # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1112        # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1113        # `backend.main.db_instance` = None; the async-job drain then crashes silently on
1114        # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1115        # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1116        # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1117        import backend.main as _bm
1118
1119        _bm.main()
1120
```

7. assistant (2026-06-22T09:08:13.441Z)

Now let me trace the compilation and stitching path:

8. user (2026-06-22T09:08:15.210Z)

```
1       import os
2       import subprocess
3       import tempfile
4       import logging
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin  # P2a: single ffmpeg/ffprobe
resolver
9
10      logger = logging.getLogger(__name__)
11
12      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
13      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
14      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
15      # backend/pipeline/stitcher/, so parents[2] == backend/.
16      _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
17
18      class VideoCompiler:
19          def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
20                       watermark: Optional[Any] = None):
21              self.output_dir = output_dir
22              self.end_padding = end_padding
23              self.begin_padding = begin_padding
24              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
25              # normalized to a dict (or None) so this module stays config-package-agnostic.
26              self.watermark = self._normalize_watermark(watermark)
27              os.makedirs(output_dir, exist_ok=True)
28
29          @staticmethod
30          def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
31              if watermark is None:
32                  return None
33              if hasattr(watermark, "model_dump"):
34                  return watermark.model_dump()
35              if isinstance(watermark, dict):
36                  return dict(watermark)
37              return None
38
39          @staticmethod
40          def _ff_quote(path: str) -> str:
41              """Make a filesystem path safe to embed inside single quotes in an ffmpeg
42              filtergraph operand (drawtext fontfile/textfile, movie= source): forward
43              slashes (Windows-friendly), escaped single quotes, AND escaped colons.
44
45              The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
46              option parser treats ':' as the option separator, so a Windows drive-letter
47              path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
48              the encode fails with "Invalid argument". This silently disabled the
49              on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
50              return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
51
52          @staticmethod
53          def _parse_time(val: Union[int, float, str]) -> float:
```

```
54              """Normalizes a timestamp value to float seconds.
55              Handles: float/int (pass-through), plain numeric strings ("12.4"),
56              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
57              This mirrors the parse_time logic used in gemini.py to ensure
58              the stitcher can always consume whatever format the indexer produced."""
59              if isinstance(val, (int, float)):
60                  return float(val)
61              if isinstance(val, str):
62                  val = val.strip()
63                  if ":" in val:
64                      # Handle MM:SS, HH:MM:SS, etc.
65                      parts = val.split(":")
66                      parts.reverse()
67                      total = 0.0
68                      for i, p in enumerate(parts):
69                          try:
70                              total += float(p) * (60 ** i)
71                          except ValueError:
72                              pass
73                      return total
74                  try:
75                      return float(val)
76                  except ValueError:
77                      logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
78                      return 0.0
79              logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
80              return 0.0
81
82          @staticmethod
83          def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
84              """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
85
86              A highlight reel must never replay the same footage. The same rally can land in
87              the segment list more than once ? e.g. two detector runs accumulated for one
88              video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
89              chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
90              twice. Merging any range that overlaps or abuts the previous one collapses true
91              duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92              Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93              """
94              parsed = []
95              for raw_s, raw_e in segments:
96                  s = VideoCompiler._parse_time(raw_s)
97                  e = VideoCompiler._parse_time(raw_e)
98                  if e > s:
99                      parsed.append((s, e))
100             parsed.sort()
101             merged: List[Tuple[float, float]] = []
102             for s, e in parsed:
103                 if merged and s <= merged[-1][1]:   # overlaps/abuts previous -> extend it
104                     merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105                 else:
106                     merged.append((s, e))
107             return merged
108
109         def _get_video_duration(self, video_path: str) -> float:
110             """Probes the video file to get its total duration in seconds.
111             Returns 0.0 if it cannot be determined."""
112             try:
113                 result = subprocess.run(
```

```
                    [
                        ffprobe_bin(), "-v", "error",
                        "-show_entries", "format=duration",
                        "-of", "default=noprint_wrappers=1:nokey=1",
                        video_path
                    ],
                    capture_output=True, text=True, check=True
                )
                return float(result.stdout.strip())
            except Exception as e:
                logger.warning(f"Could not determine video duration via ffprobe: {e}")
                return 0.0

        def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
            """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
            and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
            watermarking is disabled or has nothing to draw. The concat stage stays -c copy
            because the mark is already baked into every clip identically."""
            wm = self.watermark
            if not wm or not wm.get("enabled"):
                return None

            text = (wm.get("text") or "").strip()
            logo_path = (wm.get("logo_path") or "").strip()
            if logo_path and not os.path.exists(logo_path):
                logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
                logo_path = ""
            if not text and not logo_path:
                return None

            margin = int(wm.get("margin", 24))
            opacity = float(wm.get("opacity", 0.85))
            position = str(wm.get("position", "bottom-right"))
            # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
            dt_xy = {
                "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
                "bottom-left": f"x={margin}:y=h-th-{margin}",
                "top-right": f"x=w-tw-{margin}:y={margin}",
                "top-left": f"x={margin}:y={margin}",
            }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
            ov_xy = {
                "bottom-right": f"W-w-{margin}:H-h-{margin}",
                "bottom-left": f"{margin}:H-h-{margin}",
                "top-right": f"W-w-{margin}:{margin}",
                "top-left": f"{margin}:{margin}",
            }.get(position, f"W-w-{margin}:H-h-{margin}")

            drawtext = ""
            if text:
                ratio = float(wm.get("font_size_ratio") or 0.035)
                divisor = max(1, round(1.0 / ratio))
                # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
                tf_path = os.path.join(tmp_dir, "watermark.txt")
                with open(tf_path, "w", encoding="utf-8") as fh:
                    fh.write(text)
                # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
                # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
                font_path = (wm.get("font_path") or "").strip()
                if not font_path and _BUNDLED_FONT.exists():
                    font_path = str(_BUNDLED_FONT)
                if font_path:
```

```
                    font_opt = f"fontfile='{self._ff_quote(font_path)}'"
                else:
                    font_opt = f"font='{wm.get('font', 'Sans')}'"
                box = ""
                if wm.get("box", True):
                    box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
                drawtext = (
                    f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
                    f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
                )

            if logo_path and drawtext:
                return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
            if logo_path:
                return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
            return drawtext

        def _trim_one_segment(
            self,
            idx: int,
            raw_start: Union[int, float, str],
            raw_end: Union[int, float, str],
            segments: List[Tuple[float, float]],
            video_duration: float,
            tmp_dir: str,
            raw_video_path: str,
            watermark_filter: Optional[str],
        ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
            """Trim a single segment into its own clip file (extracted verbatim from the
            per-segment body of compile_highlights). Returns (clip_path, skip_record): on
            success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
            None and the (idx, start, end, reason) record to record in skipped_segments."""
            # Normalize timestamps to float seconds ? handles float, int,
            # plain numeric strings, and colon-separated formats like "MM:SS"
            start = self._parse_time(raw_start)
            end = self._parse_time(raw_end)
            segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"

            # Validate the segment times
            if start < 0:
                logger.warning(f"{segment_label}: start_time is negative ({start:.2f}s).
Clamping to 0.")
                start = 0.0

            if start >= end:
                logger.error(f"{segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.")
                return None, (idx, start, end, "start >= end")

            # Check if segment extends beyond video duration
            if video_duration > 0:
                if start >= video_duration:
                    logger.error(f"{segment_label}: start_time ({start:.2f}s) is beyond the
video duration ({video_duration:.2f}s). "
                                 f"This segment cannot be extracted ? the video ran out.
Skipping.")
                    return None, (idx, start, end, "start beyond video end")

                if end > video_duration:
                    original_end = end
                    end = video_duration
                    logger.warning(f"{segment_label}: end_time ({original_end:.2f}s) exceeds
video duration ({video_duration:.2f}s). "
                                   f"Clamping end_time to {video_duration:.2f}s. The segment
```

```
may be truncated.")

            clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

            # Precise sub-second trim using fast re-encoding to preserve stream headers
            padded_start = max(0.0, start - self.begin_padding)
            padded_end = end + self.end_padding

            # Clamp padded_end to video duration if known
            if video_duration > 0 and padded_end > video_duration:
                padded_end = video_duration
                logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s)
exceeds video duration. "
                            f"Clamping to {video_duration:.2f}s (end padding partially
applied).")

            trim_duration = padded_end - padded_start
            logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")

            base_cmd = [
                ffmpeg_bin(), "-y",
                "-ss", str(round(padded_start, 3)),
                "-to", str(round(padded_end, 3)),
                "-i", raw_video_path,
                "-c:v", "libx264",
                "-preset", "superfast",
                "-crf", "22",
                "-c:a", "aac",
                "-b:a", "128k",
            ]
            vf_args = ["-vf", watermark_filter] if watermark_filter else []

            # Try with the watermark first; if that encode fails (e.g. a bad font/logo
            # path), retry once WITHOUT it so a branding misconfig can never nuke the
            # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
            attempts = [vf_args, []] if vf_args else [[]]
            produced = False
            last_exc: Optional[subprocess.CalledProcessError] = None
            cmd = base_cmd + [clip_path]
            for attempt_vf in attempts:
                cmd = base_cmd + attempt_vf + [clip_path]
                try:
                    subprocess.run(cmd, capture_output=True, check=True)
                except subprocess.CalledProcessError as e:
                    last_exc = e
                    if attempt_vf and vf_args:
                        _err = (e.stderr.decode(errors="replace").strip()[-300:]
                                if getattr(e, "stderr", None) else "")
                        logger.warning(
                            f"{segment_label}: watermark encode failed; retrying without it.
"
                            f"ffmpeg: {_err or '(no stderr captured)'}"
                        )
                        continue
                    break
                if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                    produced = True
                    if not attempt_vf and vf_args:
                        logger.warning(f"{segment_label}: encoded WITHOUT the watermark (the
watermark filter failed).")
                    break
                last_exc = None  # ffmpeg succeeded but produced nothing
                break

```

```
293             if produced:
294                 return clip_path, None
295             elif last_exc is not None:
296                 stderr_msg = last_exc.stderr.decode(errors='replace').strip()
297                 logger.error(f"{segment_label}: FFmpeg trim failed.")
298                 print(f"  Command: {' '.join(cmd)}")
299                 print(f"  FFmpeg stderr: {stderr_msg[-500:]}")  # Last 500 chars to avoid
flooding
300                 return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
301             else:
302                 logger.error(f"{segment_label}: FFmpeg exited successfully but output clip
is missing or empty. Skipping.")
303                 return None, (idx, start, end, "empty output clip")
304 
305         def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
306             """
307             Extracts selected segments from a raw video and stitches them together
308             into a seamless highlights file, applying end_padding to prevent abrupt endings.
309             """
310             if not segments:
311                 raise ValueError("No highlight segments provided to compile.")
312 
313             # Never stitch the same footage twice: collapse overlapping/duplicate ranges
314             # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
315             original_count = len(segments)
316             segments = self._merge_overlapping(segments)
317             if len(segments) < original_count:
318                 logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
319                             original_count - len(segments), original_count, len(segments))
320             if not segments:
321                 raise ValueError("No valid (start < end) highlight segments after merge.")
322 
323             if not os.path.exists(raw_video_path):
324                 raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
325 
326             # Defense-in-depth: never let the output name escape output_dir (the API also
327             # validates this at the request boundary).
328             output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
329 
330             # Probe video duration so we can detect out-of-range segments
331             video_duration = self._get_video_duration(raw_video_path)
332             if video_duration > 0:
333                 logger.info(f"Source video duration: {video_duration:.2f}s")
334             else:
335                 logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
336 
337             # We will create temporary clips and stitch them using FFmpeg concat demuxer
338             temp_clips = []
339             skipped_segments = []
340 
341             # Create a temp directory within output_dir
342             with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
343                 logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
344                 # Build the (optional) watermark filtergraph once; it is applied during
every
345                 # per-clip re-encode so the mark is baked into each clip identically.
346                 watermark_filter = self._watermark_filter(tmp_dir)
347                 if watermark_filter:
348                     logger.info("[watermark] applying branding overlay to each clip.")
349                 for idx, (raw_start, raw_end) in enumerate(segments):
```

```
                    clip_path, skip_record = self._trim_one_segment(
                        idx, raw_start, raw_end, segments, video_duration,
                        tmp_dir, raw_video_path, watermark_filter,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
                    if skip_record is not None:
                        skipped_segments.append(skip_record)

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
                    logger.info("")

                if not temp_clips:
                    raise RuntimeError(
                        f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
                        f"All {len(segments)} segments were skipped. "
                        f"Check the errors above for details. "
                        f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
                    )

                logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")

                # Create a list file for FFmpeg concat demuxer
                list_file_path = os.path.join(tmp_dir, "clips.txt")
                # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
                # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
                with open(list_file_path, "w", encoding="utf-8") as f:
                    for clip in temp_clips:
                        # Escape single quotes in path
                        safe_path = clip.replace("'", "'\\''")
                        f.write(f"file '{safe_path}'\n")

                logger.info("Stitching clips together...")
                # Stitch them together without re-encoding (since they were encoded with
identical settings)
                concat_cmd = [
                    ffmpeg_bin(), "-y",
                    "-f", "concat",
                    "-safe", "0",
                    "-i", list_file_path,
                    "-c", "copy",
                    output_path
                ]

                try:
                    subprocess.run(concat_cmd, capture_output=True, check=True)

                    # Final verification
                    if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
                        raise RuntimeError(
                            f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
                            f"but output file is missing or empty: {output_path}"
                        )

                    output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
                    logger.info(f"Dynamic highlight reel saved: {output_path}
```

```
({output_size_mb:.1f} MB)")
409                     return output_path
410                 except subprocess.CalledProcessError as e:
411                     stderr_msg = e.stderr.decode(errors='replace').strip()
412                     raise RuntimeError(
413                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
414                         f"  Command: {' '.join(concat_cmd)}\n"
415                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
416                     ) from e
417
```

9. user (2026-06-22T09:08:16.618Z)

```
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py:18:class VideoCompiler:
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py:305:    def
compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
output_filename: str) -> str:
```

10. assistant (2026-06-22T09:08:19.134Z)

Now let me check the database schema for play_segments and understand what is stored:

11. user (2026-06-22T09:08:20.303Z)

```
1       import os
2       import shutil
3       import sqlite3
4       import json
5       import logging
6       from contextlib import contextmanager
7       from typing import List, Dict, Any, Optional, Tuple
8
9       from backend import billing  # pure cost math (no backend imports ? no cycle)
10
11      logger = logging.getLogger(__name__)
12
13      # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14      # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
15      # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16      # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17      # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18      SCHEMA_VERSION = 7
19
20
21      class SchemaTooNewError(RuntimeError):
22          """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24      class Database:
25          def __init__(self, db_path: str):
26              self.db_path = db_path
27              self._init_db()
28
29          @contextmanager
30          def _connect(self):
31              """Transaction scope that also CLOSES the connection.
32
33              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
34              never closes, so every call leaked its connection to GC (lingering file
35              handles on the WAL sidecars under Windows). Internal methods use this.
```

```
            """
            conn = self._get_connection()
            try:
                with conn:
                    yield conn
            finally:
                conn.close()

        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
            """Run ONE bool-returning single-statement write inside a connection scope.

            Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
            the bool-returning single-statement writers. ``error_msg`` is the caller's exact
            per-method message (already formatted with its own ids) ? logged verbatim on a
            swallowed write fault so each writer keeps its specific log text. Returns True
on
            success, False (after logging ``error_msg``) on any exception."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(sql, params)
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"{error_msg}: {e}")
                return False

        @staticmethod
        def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
            """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.

            A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
            transaction, so an interrupted v3/v4 block can leave the column added while
            ``user_version`` stays un-bumped ? and the next open would re-run the same
            ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
            future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
            ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
            ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
            """
            try:
                cursor.execute(ddl)
            except sqlite3.OperationalError as exc:
                if "duplicate column name" not in str(exc).lower():
                    raise

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
```

```
96              with self._connect() as conn:
97                  cursor = conn.cursor()
98
99                  self._create_base_tables(cursor)
100
101                 # Versioned migrations live here. PRAGMA user_version is the schema
102                 # contract ? bump it for every change and add an ordered `if version < N`
103                 # block below. Tests assert the expected version, so accidental drift fails
CI.
104                 version = cursor.execute("PRAGMA user_version").fetchone()[0]
105
106                 # P2d upgrade-safety (no-op for a fresh DB or one already at
SCHEMA_VERSION):
107                 # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
schema this
108                 #     binary can't reason about; opening it could corrupt data, so refuse
loudly.
109                 if version > SCHEMA_VERSION:
110                     raise SchemaTooNewError(
111                         f"This database (schema v{version}) was created by a newer version
of the app "
112                         f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
point it at "
113                         f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}")
114                 # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
in place; a
115                 #     failed migration could corrupt it, so snapshot it first (best-effort:
WARN, never
116                 #     block the upgrade). version 0 = a brand-new DB ? nothing to back up.
117                 if 0 < version < SCHEMA_VERSION:
118                     self._backup_before_migrate(conn, version)
119
120                 if version < 1:
121                     self._migrate_to_v1(cursor)
122
123                 if version < 2:
124                     self._migrate_to_v2(cursor)
125
126                 if version < 3:
127                     self._migrate_to_v3(cursor)
128
129                 if version < 4:
130                     self._migrate_to_v4(cursor)
131
132                 if version < 5:
133                     self._migrate_to_v5(cursor)
134
135                 if version < 6:
136                     self._migrate_to_v6(cursor)
137
138                 if version < 7:
139                     self._migrate_to_v7(cursor)
140                 # future: if version < 8: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 8
141
142                 conn.commit()
143
144         def _backup_before_migrate(self, conn: sqlite3.Connection, from_version: int) ->
None:
145             """Snapshot the DB to ``<db>.bak-v<from_version>`` before an upgrade migration
(P2d).
146
147             Checkpoints the WAL into the main file first so the copy is complete. Best-
effort: a backup
148             failure WARNS but does NOT block the upgrade (the ladder is itself crash-safe;
```

```
refusing to
149             start because a backup couldn't be written would be worse than proceeding
without one)."""
150             backup = f"{self.db_path}.bak-v{from_version}"
151             try:
152                 conn.execute("PRAGMA wal_checkpoint(FULL)")
153                 shutil.copy2(self.db_path, backup)
154                 logger.info("[DB] backed up v%s database before upgrading ? %s",
from_version, backup)
155             except Exception as exc:  # noqa: BLE001 ? the backup is a best-effort safety
net; its failure
156                 # must NEVER block the in-place upgrade (the migration ladder is itself
crash-safe). Log loud.
157                 logger.warning("[DB] could not back up the database before migrating from
v%s (%s); "
158                                "proceeding with the in-place upgrade.", from_version, exc)
159
160         def backup_database(self, dest: str) -> str:
161             """Write a consistent copy of the DB to ``dest`` (an 'export my data'
affordance, P2d).
162             Checkpoints the WAL first so ``dest`` is self-contained. Returns ``dest``."""
163             parent = os.path.dirname(os.path.abspath(dest))
164             if parent:
165                 os.makedirs(parent, exist_ok=True)
166             with self._connect() as conn:
167                 conn.execute("PRAGMA wal_checkpoint(FULL)")
168             shutil.copy2(self.db_path, dest)
169             return dest
170
171         def _create_base_tables(self, cursor):
172             """Create the videos / play_segments / events base tables (idempotent)."""
173             # Videos Table
174             cursor.execute("""
175                 CREATE TABLE IF NOT EXISTS videos (
176                     id TEXT PRIMARY KEY,
177                     filepath TEXT NOT NULL,
178                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
179                     status TEXT NOT NULL,
180                     validation_results TEXT
181                 )
182             """)
183
184             # Play Segments Table (Extensible metadata JSON)
185             cursor.execute("""
186                 CREATE TABLE IF NOT EXISTS play_segments (
187                     id INTEGER PRIMARY KEY AUTOINCREMENT,
188                     video_id TEXT NOT NULL,
189                     start_time REAL NOT NULL,
190                     end_time REAL NOT NULL,
191                     duration REAL NOT NULL,
192                     server TEXT,
193                     receiver TEXT,
194                     metadata TEXT,
195                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
196                 )
197             """)
198
199             # Events Table
200             cursor.execute("""
```

12. assistant (2026-06-22T09:08:22.681Z)

Let me continue reading the database schema to understand segment counting:

13. user (2026-06-22T09:08:24.798Z)

```
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS events (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    segment_id INTEGER NOT NULL,
                    timestamp REAL NOT NULL,
                    type TEXT NOT NULL,
                    player TEXT,
                    details TEXT,
                    FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
                )
            """)

        def _migrate_to_v1(self, cursor):
            # v1: raw candidate detections (pre-confirmation), detector-agnostic.
            # Common fields are columns; detector-specific metrics go in `stats` JSON,
            # so a new segmenter reuses this table by setting its own `detector`/`stats`.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS candidate_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    detector TEXT NOT NULL,
                    chunk_index INTEGER,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    confidence REAL,
                    stats TEXT,
                    detection_params TEXT,
                    confirmed_segment_id INTEGER,
                    human_label TEXT,
                    notes TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
                )
            """)
            cursor.execute("""
                CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                    ON candidate_segments(video_id, detector)
            """)
            cursor.execute("PRAGMA user_version = 1")

        def _migrate_to_v2(self, cursor):
            # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
            # /api/process request. `status` is the EXECUTION state of the job
            # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
            # that failed validation) lives inside `result` JSON as result["status"].
            # `result` is the full sync-era response dict (incl. report paths), so the
            # observability report is the job's artifact, per the plan.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS jobs (
                    id TEXT PRIMARY KEY,
                    video_path TEXT NOT NULL,
                    video_id TEXT,
                    segmenter TEXT,
                    player_pool TEXT,
                    max_frames INTEGER,
                    status TEXT NOT NULL DEFAULT 'queued',
                    progress TEXT,
                    error TEXT,
                    result TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    started_at TIMESTAMP,
                    finished_at TIMESTAMP
```

```
264                 )
265             """)
266             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
267             cursor.execute("PRAGMA user_version = 2")
268 
269         def _migrate_to_v3(self, cursor):
270             # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
271             # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
272             # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
273             # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
274             # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
275             # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
276             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
277             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
278             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
279             cursor.execute("PRAGMA user_version = 3")
280 
281         def _migrate_to_v4(self, cursor):
282             # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
283             # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
284             # path can scope rows to an owner. **NULL = local install = byte-identical to
285             # today** ? every existing row stays NULL and every existing query (which never
286             # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
287             # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
288             # Additive ONLY: no served-behavior change rides on this slice.
289             self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
290             self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
291             # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
292             # of index churn while making the eventual per-user lookups cheap.
293             cursor.execute(
294                 "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
295                 "WHERE user_id IS NOT NULL")
296             cursor.execute(
297                 "CREATE INDEX IF NOT EXISTS idx_candidates_user "
298                 "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
299             cursor.execute("PRAGMA user_version = 4")
300 
301         def _migrate_to_v5(self, cursor):
302             # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
303             # version): the resolved per-user `fusion_config` overlay + an INLINE
304             # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
305             # and provenance that earned it. `is_active` pins the one row the serving bridge
306             # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
307             # only persists/loads it ? NOTHING reads it on the served path yet.
308             #
309             #   user_id              ? owner of the model (NULL allowed = the local install)
310             #   version              ? monotonic per-user model version
311             #   baseline_version     ? the shared baseline this was fit against (upgrade
switch)
312             #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
313             #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
314             #   classifier_json      ? JSON LogisticRegression.to_dict() (None = config-only
model)
315             #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it was
```

```
promoted)
316             #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
317             #   is_active            ? the one resolved row for this user (partial-unique
below)
318             cursor.execute("""
319                 CREATE TABLE IF NOT EXISTS user_models (
320                     id INTEGER PRIMARY KEY AUTOINCREMENT,
321                     user_id TEXT,
322                     version INTEGER NOT NULL DEFAULT 1,
323                     baseline_version TEXT,
324                     feature_schema_hash TEXT,
325                     fusion_config TEXT,
326                     classifier_json TEXT,
327                     promotion_evidence TEXT,
328                     n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
329                     is_active INTEGER NOT NULL DEFAULT 0,
330                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
331                 )
332             """)
333             # One model version per user (NULLs compare distinct in SQLite, so the local
334             # install can still carry multiple versioned rows; the active-row guard below
335             # is what enforces a single served model).
336             cursor.execute("""
337                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
338                     ON user_models(user_id, version)
339             """)
340             # At most ONE active model per user: a partial-unique index over the active
rows.
341             # (SQLite treats NULL user_id rows as distinct, so the local install's single
342             # active row is still guarded ? there is one local user.)
343             cursor.execute("""
344                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
345                     ON user_models(user_id) WHERE is_active = 1
346             """)
347             cursor.execute("PRAGMA user_version = 5")
348
349         def _migrate_to_v6(self, cursor):
350             # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
351             # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
352             # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
353             # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
354             # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNs are
355             # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
356             for col, typ in (
357                 ("owner_id", "TEXT"),               # who pays (the non-owner user / tenant)
358                 ("compute_backend", "TEXT"),        # local_gpu | cloud_run | ...
359                 ("gpu_type", "TEXT"),               # L4 | T4 | ... (selects the per-second
GPU rate)
360                 ("gpu_active_seconds", "REAL"),
361                 ("wall_seconds", "REAL"),
362                 ("bytes_out", "INTEGER"),           # egress
363                 ("gemini_cost_usd", "REAL"),        # provider-reported, already USD
364                 ("pricebook_version", "TEXT"),
365                 ("cost_usd", "REAL"),               # computed total (USD = source of truth)
366                 ("cost_breakdown_json", "TEXT"),    # {gpu_seconds: .., egress_gb: ..,
gemini_usd: ..}
367             ):
368                 self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
```

```
{typ}")
            # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
                           "WHERE owner_id IS NOT NULL")
            # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
            # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS pricebook (
                    version TEXT NOT NULL,
                    resource TEXT NOT NULL,
                    gpu_type TEXT NOT NULL DEFAULT '',
                    unit_price_usd REAL NOT NULL DEFAULT 0.0,
                    currency TEXT NOT NULL DEFAULT 'USD',
                    effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
            """)
            cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
                           "ON pricebook(version, resource, gpu_type)")
            # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
            # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
            # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
            for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
                                       ("wall_seconds", ""), ("egress_gb", "")):
                cursor.execute(
                    "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
                    "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource,
gpu_type))
            cursor.execute("PRAGMA user_version = 6")

        def _migrate_to_v7(self, cursor):
            # v7: record the UI LOCALE a job was submitted in (the language the SPA was
rendered in,
            # e.g. "kn", "hi-IN"). NULLABLE ? NULL = unrecorded = byte-identical to today
(CLI / older
            # clients / pre-v7 rows). Enables PMF analytics ("which language markets use the
tool",
            # count_jobs_by_locale) and pairs with the localized reel watermark (the SPA
sends the same
            # locale to /api/compile). Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN locale TEXT")
            cursor.execute("PRAGMA user_version = 7")

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            return self._execute_write(
```

```
                "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
                "ON CONFLICT(id) DO UPDATE SET "
                "filepath=excluded.filepath, status=excluded.status, "
                "validation_results=excluded.validation_results",
                (video_id, filepath, status, json.dumps(validation_results)),
                "Failed to add video",
            )

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
```

```
466                     )
467                     conn.commit()
468                     return cursor.lastrowid
469             except Exception as e:
470                 logger.error(f"Failed to add play segment: {e}")
471                 return None
472 
473         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474             return self._execute_write(
475                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477                 "Failed to add event",
478             )
479 
480         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
482                                   stats: Optional[Dict[str, Any]] = None,
483                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
484             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
485             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
486             try:
487                 with self._connect() as conn:
488                     cursor = conn.cursor()
489                     cursor.execute(
490                         """INSERT INTO candidate_segments
491                            (video_id, detector, chunk_index, start_time, end_time, duration,
492                             confidence, stats, detection_params)
493                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
494                         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
495                          confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
496                     )
497                     conn.commit()
498                     return cursor.lastrowid
499             except Exception as e:
```

14. user (2026-06-22T09:08:27.004Z)

```
545:    def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
564:    def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
642:    def count_jobs_by_locale(self) -> Dict[str, int]:
```

15. assistant (2026-06-22T09:08:30.322Z)

Now let me read the query_play_segments method to understand the full pipeline:

16. user (2026-06-22T09:08:32.048Z)

```
564         def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
565             """
566             Dynamically queries play segments based on filters:
567             - duration_min: float
568             - server: string
569             - receiver: string
570             - event_type: string (e.g. smash, foul)
```

```
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) -------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None,
                       owner_id: Optional[str] = None,
                       locale: Optional[str] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
            (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
```

```
(default).
632             ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded."""
633             return self._execute_write(
634                 "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
635                 "owner_id, locale) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?)",
636                 (job_id, video_path, segmenter,
637                  json.dumps(player_pool) if player_pool is not None else None,
638                  max_frames, json.dumps(policy) if policy is not None else None, owner_id,
locale),
639                 f"Failed to create job {job_id}",
640             )
641
642         def count_jobs_by_locale(self) -> Dict[str, int]:
643             """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
644             CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
645             with self._connect() as conn:
646                 rows = conn.cursor().execute(
647                     "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
648                     "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
649                 ).fetchall()
650             return {str(r[0]): int(r[1]) for r in rows}
651
652         def mark_job_running(self, job_id: str) -> bool:
653             return self._execute_write(
654                 "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
655                 "progress='starting' WHERE id = ?", (job_id,),
656                 f"Failed to mark job {job_id} running",
657             )
658
659         def set_job_progress(self, job_id: str, progress: str) -> bool:
660             return self._execute_write(
661                 "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
662                 f"Failed to set progress for job {job_id}",
663             )
```

17. assistant (2026-06-22T09:08:38.545Z)

Now let me check how the regression test should work by searching for any existing regression or nightly test patterns:

18. user (2026-06-22T09:08:40.285Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/per_user_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/nightly_baseline.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/run_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.ps1
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/register_nightly_task.ps1
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_eval_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_nightly_regression.py
C:/Users/avidu/Projects/badminton-highlight-
```

indexer/.claude/worktrees/nightly-e2e/tests/test_per_user_regression.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/tests/test_served_gate_a_regression.py

19. assistant (2026-06-22T09:08:42.419Z)

Perfect! Let me check the existing nightly regression script:

20. user (2026-06-22T09:08:43.654Z)

```
1       """Nightly rally-quality regression tripwire.
2
3       Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4       seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5       the headline metrics against a FROZEN baseline
(``eval_baselines/nightly_baseline.json``).
6       Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
7       over-segmentation-guard regression.
8
9       This is the owner's dual-eval-set discipline made automatic (``memory/eval-dual-set-
regression``
10      + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11      night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
12      does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13      no WASB re-extraction (those are cached and unchanged night to night).
14
15      Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17      * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18        to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19        SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20      * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21        ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22        the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23      * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24        permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25        win must keep standing its ground, not be taken at launch-time value).
26
27      Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28      outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29      baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30      whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31      has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32      ``scripts/nightly_regression.ps1``.)
33
34      Usage
35      -----
36      Snapshot the baseline from current HEAD (after an intended improvement or a corpus
```

```
change):
37
38          python scripts/nightly_regression.py --update-baseline
39
40      Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42          python scripts/nightly_regression.py
43
44      Exit codes
45      ----------
46      * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47      * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48        over-seg guard increased). Takes precedence over a stale baseline.
49      * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50      * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51      * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52        but no regression was found on the comparable intersection.
53      """
54      from __future__ import annotations
55
56      import argparse
57      import datetime as _dt
58      import json
59      import os
60      import sys
61      from dataclasses import dataclass, field, replace
62      from typing import Any, Dict, List, Optional, Tuple
63
64      # ---------------------------------------------------------------------------- #
65      # Paths (resolved relative to the repo root, so the script is cwd-robust).
66      # ---------------------------------------------------------------------------- #
67      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68      if REPO_ROOT not in sys.path:
69          sys.path.insert(0, REPO_ROOT)  # importable as `python
scripts/nightly_regression.py` from anywhere
70      DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71      DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72      DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73      DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74      DEFAULT_IOU = 0.5
75
76      # ---------------------------------------------------------------------------- #
77      # What we gate on (a tripwire ? focused, not every metric).
78      #   * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
79      #   * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
80      #   * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
81      #     (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82      # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83      # ---------------------------------------------------------------------------- #
84      GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85      GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86      PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88      #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89      AGG_SNAPSHOT: Tuple[str, ...] = (
90          "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91          "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
"dend_abs_median",
92      )
93      #: Per-video metrics snapshotted into the baseline.
```

```
94      PV_SNAPSHOT: Tuple[str, ...] = (
95          "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
"num_pred", "num_gt",
96      )
97
98      DEFAULT_METRIC_TOL = 0.005   # F1-like drop that counts as a regression (absorbs float
jitter)
99      DEFAULT_RATE_TOL = 0.010     # merge_rate rise that counts as a regression
100     DEFAULT_OVER_SEG_TOL = 0     # per-video split/merge increase allowed (strict)
101
102
103     # ---------------------------------------------------------------------------- #
104     # The two tracked configs.
105     # ---------------------------------------------------------------------------- #
106     @dataclass(frozen=True)
107     class NightlyConfig:
108         """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109
110         name: str
111         preset: Any              # backend.eval.windowing.WindowingPreset
112         min_crossings: int = 0
113         note: str = ""
114
115
116     def tracked_configs() -> List[NightlyConfig]:
117         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).
118
119         All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
120         min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
121
122         * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
123           point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
124           DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
125           knob.
126         * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
127           the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
128           what we expect?" stays answerable even if a future change moves DEFAULT.
129         * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
130           the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
131           dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
132           launch-time value). It is the reference DEFAULT used to be before the flip."""
133         from backend.eval.windowing import SERVED
134
135         default = NightlyConfig(
136             name="default", preset=SERVED, min_crossings=0,
137             note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
138         )
139         milestone = NightlyConfig(
140             name="milestone",
141             preset=replace(SERVED, name="milestone", max_window_duration=6.0,
142                            inactivity_end=1.5, inactivity_rally_thresh=0.20,
143                            inactivity_min_density=0.30),
144             min_crossings=0,
145             note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
```

```
default post-#204).",
        )
        baseline_off = NightlyConfig(
            name="baseline_off",
            preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
hard_cap=False,
                           inactivity_end=0.0, inactivity_rally_thresh=0.08,
inactivity_min_density=0.34,
                           blob_fallback=False, blob_factor=4.0),
            min_crossings=0,
            note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
        )
        return [default, milestone, baseline_off]


    # ---------------------------------------------------------------------- #
    # Tolerances bundle.
    # ---------------------------------------------------------------------- #
    @dataclass(frozen=True)
    class Tolerances:
        metric_tol: float = DEFAULT_METRIC_TOL
        rate_tol: float = DEFAULT_RATE_TOL
        over_seg_tol: int = DEFAULT_OVER_SEG_TOL

        def to_dict(self) -> Dict[str, Any]:
            return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,
                    "over_seg_tol": self.over_seg_tol}


    # ---------------------------------------------------------------------- #
    # Pure summary + comparison core (no I/O ? unit-testable).
    # ---------------------------------------------------------------------- #
    def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
        """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.

        Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
        which is seeded but irrelevant to the gate) and per-video RAW metric values."""
        agg = eval_result.get("aggregate", {}) or {}
        aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
        per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
        return {
            "n": int(agg.get("n", len(eval_result.get("rows", [])))),
            "preset": eval_result.get("preset"),
            "min_crossings": eval_result.get("min_crossings", 0),
            "iou": eval_result.get("iou", DEFAULT_IOU),
            "aggregate": aggregate,
            "per_video": per_video,
        }


    @dataclass
    class Finding:
        """One comparison outcome line (regression / improvement / stale note)."""

        config: str
        scope: str          # "aggregate" | "per_video" | "config"
        metric: str
        kind: str           # "regression" | "improvement" | "stale"
        baseline: Optional[float] = None
        current: Optional[float] = None
        video: Optional[str] = None
```

```
204
205         @property
206         def delta(self) -> Optional[float]:
207             if self.baseline is None or self.current is None:
208                 return None
209             return self.current - self.baseline
210
211         def message(self) -> str:
212             loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
f"{self.config}/{self.metric}"
213             if self.kind == "stale":
214                 return f"[STALE] {self.config}: {self.metric}"
215             d = self.delta
216             ds = f"{d:+.4f}" if d is not None else "?"
217             b = "?" if self.baseline is None else f"{self.baseline:.4f}"
218             c = "?" if self.current is None else f"{self.current:.4f}"
219             tag = "REGRESSION" if self.kind == "regression" else "improvement"
220             return f"[{tag}] {loc}: {b} ? {c} ({ds})"
221
222
223     @dataclass
224     class ConfigComparison:
225         config: str
226         status: str = "ok"            # "ok" | "regression" | "stale"
227         findings: List[Finding] = field(default_factory=list)
228         added_videos: List[str] = field(default_factory=list)
229         removed_videos: List[str] = field(default_factory=list)
230         aggregate_gated: bool = True  # False when preset/corpus changed ? aggregate not
comparable
231
232         @property
233         def regressions(self) -> List[Finding]:
234             return [f for f in self.findings if f.kind == "regression"]
235
236         @property
237         def improvements(self) -> List[Finding]:
238             return [f for f in self.findings if f.kind == "improvement"]
239
240
241     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
242         """Are the two stored preset dicts the same windowing definition? (name is
cosmetic)."""
243         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
244             return base_preset == cur_preset
245         keys = set(base_preset) | set(cur_preset)
246         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
247
248
249     def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
250                        tols: Tolerances) -> ConfigComparison:
251         """Compare one config's current summary to its baseline summary.
252
253         * If the preset definition or the net-crossings gate changed, the whole config is
STALE
254           (numbers aren't comparable) ? flag it, skip gating.
255         * If the corpus (video set) changed, the aggregate is not comparable (different
videos) ?
256           skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
the
257           INTERSECTION (a code regression on a shared video must still trip the wire).
258         """
259         cmp = ConfigComparison(config=name)
260
261         # 1) Config-definition drift ? stale, not gateable.
262         if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
```

```
                or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
            cmp.status = "stale"
            cmp.aggregate_gated = False
            cmp.findings.append(Finding(config=name, scope="config", metric="preset",
kind="stale"))
            return cmp

        base_pv = base_sum.get("per_video", {})
        cur_pv = cur_sum.get("per_video", {})
        base_names, cur_names = set(base_pv), set(cur_pv)
        cmp.added_videos = sorted(cur_names - base_names)
        cmp.removed_videos = sorted(base_names - cur_names)
        corpus_changed = bool(cmp.added_videos or cmp.removed_videos)

        # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
        if corpus_changed:
            cmp.aggregate_gated = False
            cmp.status = "stale"
            cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
kind="stale"))
        else:
            base_agg = base_sum.get("aggregate", {})
            cur_agg = cur_sum.get("aggregate", {})
            for m in GATED_HIGHER:
                b, c = base_agg.get(m), cur_agg.get(m)
                if b is None or c is None:
                    continue
                if c < b - tols.metric_tol:
                    cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
                elif c > b + tols.metric_tol:
                    cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
            for m in GATED_LOWER:
                b, c = base_agg.get(m), cur_agg.get(m)
                if b is None or c is None:
                    continue
                if c > b + tols.rate_tol:
                    cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
                elif c < b - tols.rate_tol:
                    cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))

        # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
        for vid in sorted(base_names & cur_names):
            for m in PER_VIDEO_GUARD:
                b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
                if b is None or c is None:
                    continue
                if c > b + tols.over_seg_tol:
                    cmp.findings.append(Finding(name, "per_video", m, "regression",
float(b), float(c), video=vid))

        if cmp.regressions:
            cmp.status = "regression"
        elif cmp.status != "stale":
            cmp.status = "ok"
        return cmp


    @dataclass
    class NightlyResult:
        comparisons: List[ConfigComparison] = field(default_factory=list)
        skipped: Dict[str, List[str]] = field(default_factory=dict)  # config -> manifest
videos not scored

```

```
        @property
        def has_regression(self) -> bool:
            return any(c.status == "regression" for c in self.comparisons)

        @property
        def has_stale(self) -> bool:
            return any(c.status == "stale" for c in self.comparisons)

        def overall_status(self) -> str:
            if self.has_regression:
                return "REGRESSION"
            if self.has_stale:
                return "STALE"
            return "PASS"

        def exit_code(self) -> int:
            if self.has_regression:
                return 1
            if self.has_stale:
                return 4
            return 0


    def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
                    tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
        """Compare every tracked config's current summary to the baseline summary."""
        res = NightlyResult(skipped=skipped or {})
        base_cfgs = baseline.get("configs", {})
        cur_cfgs = current.get("configs", {})
        for name in cur_cfgs:
            if name not in base_cfgs:
                cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
                cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
                res.comparisons.append(cmp)
                continue
            res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
        return res


    # ---------------------------------------------------------------------- #
    # Report formatting (pure).
    # ---------------------------------------------------------------------- #
    def _fmt(v: Optional[float]) -> str:
        return "?" if v is None else f"{v:.4f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, tols: Tolerances, date: str) -> str:
        L: List[str] = []
        status = result.overall_status()
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
        L.append(f"# Nightly rally-quality regression ? {date}")
        L.append("")
        L.append(f"**Status: {badge}**  ·  exit code {result.exit_code()}")
        L.append("")
        L.append(f"- HEAD: `{current.get('git_commit', '?')}`  ·  baseline: "
                 f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')})")
        L.append(f"- Corpus manifest: `{current.get('manifest', '?')}`  ·
IoU={current.get('iou', DEFAULT_IOU)}")
        L.append(f"- Tolerances: metric ±{tols.metric_tol}, rate ±{tols.rate_tol}, "
```

```
                 f"per-video over-seg +{tols.over_seg_tol}")
        L.append("")

        # Up-front regression / stale summary.
        regs = [f for c in result.comparisons for f in c.regressions]
        if regs:
            L.append("## ? Regressions")
            L.append("")
            for f in regs:
                L.append(f"- {f.message()}")
            L.append("")
        if result.has_stale:
            L.append("## ? Stale / not comparable")
            L.append("")
            for c in result.comparisons:
                if c.status == "stale":
                    if c.added_videos:
                        L.append(f"- `{c.config}`: corpus added {c.added_videos}")
                    if c.removed_videos:
                        L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
                    if not c.added_videos and not c.removed_videos:
                        L.append(f"- `{c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
            L.append("- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
            L.append("")

        # Skipped (no silent caps).
        skipped_any = {k: v for k, v in result.skipped.items() if v}
        if skipped_any:
            L.append("## ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
            L.append("")
            for cfg, vids in skipped_any.items():
                L.append(f"- `{cfg}`: {vids}")
            L.append("")

        # Per-config detail.
        cur_cfgs = current.get("configs", {})
        base_cfgs = baseline.get("configs", {})
        for cmp in result.comparisons:
            cur = cur_cfgs.get(cmp.config, {})
            base = base_cfgs.get(cmp.config, {})
            L.append(f"## Config `{cmp.config}` ? {cur.get('note', '')}")
            L.append("")
            L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})_")
            L.append("")
            cagg = cur.get("aggregate", {})
            bagg = base.get("aggregate", {})
            L.append("| metric | baseline | current | ? | gated |")
            L.append("|---|---|---|---|---|")
            for m in AGG_SNAPSHOT:
                b, c = bagg.get(m), cagg.get(m)
                d = (c - b) if (b is not None and c is not None) else None
                gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
                L.append(f"| {m} | {_fmt(b)} | {_fmt(c)} | {_fmt(d)} | {gated} |")
            L.append("")
            # Per-video over-seg guard table.
            cpv = cur.get("per_video", {})
            bpv = base.get("per_video", {})
            names = sorted(set(cpv) | set(bpv))
            if names:
                L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
```

```
                L.append("|---|---|---|---|")
                for v in names:
                    cb, cc = bpv.get(v, {}), cpv.get(v, {})
                    tf = f"{_fmt(cb.get('tol_f1'))}?{_fmt(cc.get('tol_f1'))}"
                    sp = f"{cb.get('split_count', '?')}?{cc.get('split_count', '?')}"
                    mg = f"{cb.get('merge_count', '?')}?{cc.get('merge_count', '?')}"
                    flag = " ?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""
                    L.append(f"| {v}{flag} | {tf} | {sp} | {mg} |")
                L.append("")
            imps = cmp.improvements
            if imps:
                L.append("_Improvements over baseline (consider `--update-baseline` to
ratchet the floor up):_")
                for f in imps:
                    L.append(f"- {f.message()}")
                L.append("")

        L.append("---")
        L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change "
                 "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
        return "\n".join(L)


    # ---------------------------------------------------------------------- #
    # I/O shell.
    # ---------------------------------------------------------------------- #
    def _git_commit() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:
            return "?"


    def _display_path(path: str) -> str:
        """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
        in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
        Metadata only ? the comparison never reads it."""
        norm = path.replace("\\", "/")
        parent = os.path.basename(os.path.dirname(norm))
        if parent == "output":
            return "output/" + os.path.basename(norm)
        try:
            rel = os.path.relpath(path, REPO_ROOT)
            if not rel.startswith(".."):
                return rel.replace("\\", "/")
        except ValueError:
            pass
        return norm


    def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
                   iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
        """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).

        ``snapshot`` is the serialisable structure stored as / compared to the baseline.
``skipped``
```

```
        lists manifest videos NOT scored (missing trajectory or empty GT) per config ? surfaced in
        the report so the corpus can't silently shrink."""
        from backend.eval import rally_seg_eval

        golden = rally_seg_eval.load_golden(manifest, features_dir)
        manifest_names = [v["name"] for v in golden]
        if not golden:
            raise RuntimeError(f"no golden videos loaded from {manifest}")

        snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou, "configs": {}}
        skipped: Dict[str, List[str]] = {}
        scored_any = False
        for cfg in configs:
            result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou, min_crossings=cfg.min_crossings)
            summary = summarize_run(result)
            summary["note"] = cfg.note
            snapshot["configs"][cfg.name] = summary
            scored = set(summary["per_video"])
            skipped[cfg.name] = [n for n in manifest_names if n not in scored]
            scored_any = scored_any or bool(scored)
        if not scored_any:
            raise RuntimeError(
                f"scored 0 videos for every config ? are the trajectory CSVs in {manifest} present "
                f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?")
        return snapshot, skipped


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_commit()
        out["tolerances"] = tols.to_dict()
        out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
                       "`python scripts/nightly_regression.py --update-baseline` after an intended "
                       "improvement or a corpus change. Tied to the LOCAL golden corpus (not committed).")
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire (offline, CPU).")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
        p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
                       help="dir holding <name>_golden_features.json")
        p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON path")
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
```

```
are written")
        p.add_argument("--iou", type=float, default=DEFAULT_IOU)
        p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
                       help="F1-like aggregate drop that counts as a regression")
        p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
                       help="merge_rate rise that counts as a regression")
        p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
                       help="per-video split/merge-count increase allowed before it's a
regression")
        p.add_argument("--update-baseline", action="store_true",
                       help="snapshot the current numbers as the new baseline instead of
checking")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_regression")
        args = build_parser().parse_args(argv)
        tols = Tolerances(metric_tol=args.metric_tol, rate_tol=args.rate_tol,
over_seg_tol=args.over_seg_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s ? the golden corpus must be present locally.",
args.manifest)
            return 2

        try:
            snapshot, skipped = run_corpus(tracked_configs(), args.manifest,
args.features_dir, args.iou)
        except RuntimeError as e:
            log.error("%s", e)
            return 2
        snapshot["git_commit"] = _git_commit()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot, tols)
            log.info("baseline snapshot written to %s", args.baseline)
            for cfg, s in snapshot["configs"].items():
                agg = s["aggregate"]
                log.info("  %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
                         cfg, s["n"], agg.get("tol_f1") or 0.0, agg.get("f1@0.5") or 0.0,
                         agg.get("merge_rate") or 0.0)
            return 0

        baseline = load_baseline(args.baseline)
        if baseline is None:
            log.error("no baseline at %s ? snapshot one with --update-baseline (then commit
it).", args.baseline)
            return 3

        result = compare_all(baseline, snapshot, tols, skipped)
        report = format_report(baseline, snapshot, result, tols, date)

        if not args.no_report:
            os.makedirs(args.report_dir, exist_ok=True)
            md_path = os.path.join(args.report_dir, f"{date}.md")
            json_path = os.path.join(args.report_dir, f"{date}.json")
```

```
611             with open(md_path, "w", encoding="utf-8") as fh:
612                 fh.write(report + "\n")
613             with open(json_path, "w", encoding="utf-8") as fh:
614                 json.dump({"date": date, "status": result.overall_status(),
615                            "exit_code": result.exit_code(), "current": snapshot,
616                            "regressions": [f.message() for c in result.comparisons for f in
c.regressions]},
617                           fh, indent=2, sort_keys=True)
618             log.info("report written to %s", md_path)
619
620         print(report)
621         code = result.exit_code()
622         if code == 1:
623             log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                       sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625         elif code == 4:
626             log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627         else:
628             log.info("nightly regression check PASSED")
629         return code
630
631
632     if __name__ == "__main__":
633         raise SystemExit(main())
634
```

21. assistant (2026-06-22T09:08:46.213Z)

Now let me check if there's a simpler e2e test or script that runs a video through the pipeline:

22. user (2026-06-22T09:08:47.219Z)

```
1       """CLI: regression-test the production tool against golden-set ground truth.
2
3       Obtains ground truth from an annotation provider (a tier), scores the video's
4       stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5       detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7       Examples
8       --------
9       Snapshot the current numbers as the baseline (first run / after a real improvement):
10          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12      Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14      """
15      import os
16      import sys
17      import json
18      import argparse
19      import logging
20
21      from backend.infrastructure.database import Database
22      from backend.config import load_config
23      from backend.annotations import annotation_registry
24      from backend.eval import regression
25
26
27      def _default_db_path() -> str:
28          cfg = load_config()
29          return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
```

```
    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set ground truth.")
        p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose predictions to score.")
        p.add_argument("--tier", default="file",
                       help=f"Annotation provider tier. Available: {annotation_registry.list_available()}")
        p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for the 'file' tier).")
        p.add_argument("--stage", choices=("candidates", "final"), default="final")
        p.add_argument("--detector", default=None, help="Restrict candidate scoring to one detector.")
        p.add_argument("--iou", type=float, default=0.5)
        p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
        p.add_argument("--db", default=None)
        p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
        p.add_argument("--update-baseline", action="store_true",
                       help="Snapshot the current numbers as the new baseline instead of checking.")
        p.add_argument("--allow-missing-baseline", action="store_true",
                       help="Treat a missing baseline as a pass (exit 0) instead of a gate error (exit 3).")
        p.add_argument("--json", dest="json_out", default=None)
        return p


    def main(argv=None) -> int:
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        args = build_parser().parse_args(argv)

        db_path = args.db or _default_db_path()
        if not os.path.exists(db_path):
            print(f"ERROR: database not found at {db_path}. Run the pipeline first.", file=sys.stderr)
            return 2
        db = Database(db_path)

        result = regression.regress_with_provider(
            db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
            stage=args.stage, iou_threshold=args.iou, detector=args.detector,
            tolerance=args.tolerance, baseline_path=args.baseline,
        )

        print(regression.format_result(result))

        if args.update_baseline:
            regression.save_baseline(result.video_id, result.stage, result.precision,
                                     result.recall, result.f1, path=args.baseline,
                                     iou_threshold=result.iou_threshold,
detector=args.detector,
                                     gt_fingerprint=result.gt_hash)
            print(f"Baseline updated in {args.baseline}.")

        if args.json_out:
            with open(args.json_out, "w") as f:
                json.dump(result.as_dict(), f, indent=2)
            print(f"JSON written to {args.json_out}")

        # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
        if result.regressed:
            return 1
        if not result.has_baseline and not args.update_baseline and not args.allow_missing_baseline:
```

```
86              print("ERROR: no baseline to compare against ? the gate cannot verify anything.
"
87                    "Snapshot one with --update-baseline (then commit it), or pass "
88                    "--allow-missing-baseline to accept this run.", file=sys.stderr)
89              return 3
90          return 0
91
92
93      if __name__ == "__main__":
94          raise SystemExit(main())
95
```

23. assistant (2026-06-22T09:08:49.621Z)

Let me now search for existing golden fixtures or test videos to understand the structure:

24. user (2026-06-22T09:08:51.609Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/experiment_leaderboard.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/clean_single_rally/expected.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/clean_single_rally/provenance.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/manifest.json
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-
e2e/eval_baselines/fixtures/multi_rally_with_deadtime/expected.json
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-
e2e/eval_baselines/fixtures/multi_rally_with_deadtime/provenance.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/occlusion_break/expected.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/occlusion_break/provenance.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/gemini_wrapper_2026-06-10.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/heuristic_lovo_n6.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/nightly_baseline.json
```

25. assistant (2026-06-22T09:08:54.226Z)

Now let me trace the full pipeline for creating/stitching reels and see where the count is
tracked:

26. user (2026-06-22T09:08:54.979Z)


```
1       [
2         {
3           "fps": 30.0,
4           "frame_width": 1280.0,
5           "kind": "synthetic",
6           "paths": {
7             "dir": "clean_single_rally",
8             "expected": "clean_single_rally/expected.json",
9             "labels": "clean_single_rally/labels.csv",
10            "provenance": "clean_single_rally/provenance.json",
11            "trajectory": "clean_single_rally/trajectory.csv"
12          },
13          "schema_version": 1,
14          "slug": "clean_single_rally",
15          "windowing": {
16            "max_win": 0.0,
```

```
            "mg": 2.0,
            "mwd": 2.0,
            "vt": 0.02
          }
        },
        {
          "fps": 30.0,
          "frame_width": 1280.0,
          "kind": "synthetic",
          "paths": {
            "dir": "multi_rally_with_deadtime",
            "expected": "multi_rally_with_deadtime/expected.json",
            "labels": "multi_rally_with_deadtime/labels.csv",
            "provenance": "multi_rally_with_deadtime/provenance.json",
            "trajectory": "multi_rally_with_deadtime/trajectory.csv"
          },
          "schema_version": 1,
          "slug": "multi_rally_with_deadtime",
          "windowing": {
            "max_win": 0.0,
            "mg": 2.0,
            "mwd": 2.0,
            "vt": 0.02
          }
        },
        {
          "fps": 30.0,
          "frame_width": 1280.0,
          "kind": "synthetic",
          "paths": {
            "dir": "occlusion_break",
            "expected": "occlusion_break/expected.json",
            "labels": "occlusion_break/labels.csv",
            "provenance": "occlusion_break/provenance.json",
            "trajectory": "occlusion_break/trajectory.csv"
          },
          "schema_version": 1,
          "slug": "occlusion_break",
          "windowing": {
            "max_win": 0.0,
            "mg": 2.0,
            "mwd": 2.0,
            "vt": 0.02
          }
        }
      ]

```


27. assistant (2026-06-22T09:08:57.187Z)

Let me check one of the expected.json files to see what segments are expected:

28. user (2026-06-22T09:08:57.907Z)


```
      {
        "candidates": [
          {
            "end": 3.966667,
            "shuttle": {
              "active_ratio": 1.008475,
              "duration": 3.933333,
              "mean_velocity": 1.27678,
              "net_crossings": 7,
              "peak_velocity": 1.970041
            },
```

```
            "start": 0.033333
          }
        ],
        "control_segments": [
          [
            0.033333,
            3.966667
          ]
        ],
        "gt_hash": "6c93801fd607",
        "iou": 0.5,
        "schema_version": 1,
        "score": {
          "f1": 1.0,
          "false_negatives": 0,
          "false_positives": 0,
          "iou_threshold": 0.5,
          "mean_end_error": 0.033333,
          "mean_iou": 0.983333,
          "mean_start_error": 0.033333,
          "num_gt": 1,
          "num_pred": 1,
          "precision": 1.0,
          "recall": 1.0,
          "true_positives": 1
        },
        "slug": "clean_single_rally"
      }

```


29. assistant (2026-06-22T09:09:00.399Z)

Now let me look at the config to understand stitching settings and how /api/compile works:

30. user (2026-06-22T09:09:02.215Z)


```
.claude\worktrees\nightly-e2e\backend\config\models.py-185-    window_blob_factor: float = 4.0
.claude\worktrees\nightly-e2e\backend\config\models.py-186-    log_yolo_candidates: bool = True
.claude\worktrees\nightly-e2e\backend\config\models.py-187-    store_yolo_candidates: bool =
True
.claude\worktrees\nightly-e2e\backend\config\models.py:188:    ai_handoff_padding: float = 2.0
.claude\worktrees\nightly-e2e\backend\config\models.py-189-    ai_max_workers: int = 5
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-190-    # Width (px) the gemini
segmenter re-encodes chunks to before upload. Stream-copied
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-191-    # chunks of a high-
bitrate (4K) source blow past the provider upload cap
--
.claude\worktrees\nightly-e2e\backend\config\models.py-238-class WatermarkConfig(BaseModel):
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-239-    """Branding overlay
burned into each highlight clip during the per-clip re-encode
.claude\worktrees\nightly-e2e\backend\config\models.py-240-
(backend\pipeline\stitcher\compiler.py). **On by default** ? set `enabled: false`
.claude\worktrees\nightly-e2e\backend\config\models.py:241:    (under `stitching.watermark` in
config.json) to turn it off. The text is fully
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-242-    configurable. The concat
stage stays stream-copy (-c copy)."""
.claude\worktrees\nightly-e2e\backend\config\models.py-243-    enabled: bool = True
.claude\worktrees\nightly-e2e\backend\config\models.py-244-    text: str = "Generated by
Khelsutra.guru"
--
.claude\worktrees\nightly-e2e\backend\config\models.py-252-    position: Literal["bottom-right",
```

```
"bottom-left", "top-right", "top-left"] = "bottom-right"
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-253-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-254-
.claude\worktrees\nightly-e2e\backend\config\models.py:255:class StitchingConfig(BaseModel):
.claude\worktrees\nightly-e2e\backend\config\models.py:256:    begin_padding: float = 0.5
.claude\worktrees\nightly-e2e\backend\config\models.py:257:    end_padding: float = 0.5
.claude\worktrees\nightly-e2e\backend\config\models.py-258-    use_cache: bool = False
.claude\worktrees\nightly-e2e\backend\config\models.py:259:    min_shot_count: int = 1
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-260-    # Long-rally reel filter
(seconds; 0 = OFF). Drop detected rally pairs whose duration
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-261-    # (end ? start) is below
this from the highlight reel, BEFORE compiling. A cleaner
.claude\worktrees\nightly-e2e\backend\config\models.py:262:    # reel-selection knob than
`min_shot_count`: shot_count is unlabeled / "Unknown" on the
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-263-    # fully-local no-AI path,
whereas duration is derived from the rally's own start/end. The
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-264-    # local detector over-
fires and its false positives are mostly SHORT (motion blips,
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\nightly-e2e\backend\config\models.py-265-    # background-court
fragments), so this keeps the reel to the eye-catching long rallies.
--
.claude\worktrees\nightly-e2e\backend\config\models.py-400-    validation: ValidationConfig =
Field(default_factory=ValidationConfig)
.claude\worktrees\nightly-e2e\backend\config\models.py-401-    indexing: IndexingConfig =
Field(default_factory=IndexingConfig)
.claude\worktrees\nightly-e2e\backend\config\models.py-402-    output: OutputConfig =
Field(default_factory=OutputConfig)
.claude\worktrees\nightly-e2e\backend\config\models.py:403:    stitching: StitchingConfig =
Field(default_factory=StitchingConfig)
.claude\worktrees\nightly-e2e\backend\config\models.py-404-    security: SecurityConfig =
Field(default_factory=SecurityConfig)
.claude\worktrees\nightly-e2e\backend\config\models.py-405-    logging: LoggingConfig =
Field(default_factory=LoggingConfig)
.claude\worktrees\nightly-e2e\backend\config\models.py-406-    storage: StorageConfig =
Field(default_factory=StorageConfig)
```

31. assistant (2026-06-22T09:09:05.931Z)

Perfect. Now let me trace through the full pipeline by checking the main.py compile_highlights to understand the complete flow:

32. user (2026-06-22T09:09:06.936Z)

```
705     @app.post("/api/compile")
706     def compile_highlights(req: CompileRequest):
707         video = db_instance.get_video(req.video_id)
708         if not video:
709             raise HTTPException(status_code=404, detail="Indexed video record not found.")
710
711         # Retrieve matching play segments from db
712         all_segments = db_instance.query_play_segments(req.video_id, {})
713         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
714
715         if not matched_segments:
716             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
717
718         # Reject path traversal / absolute output names (os.path.join would otherwise let an
```

```
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "
f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
        # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
        # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        localized_watermark = localize_watermark(stitching.watermark, req.locale)
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
watermark=localized_watermark)
        try:
            # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
            # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
            # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
            # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
            # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
            local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
            # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
```

```
767             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
768             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
769             return {"status": "success", "filepath": out_path, "download_url": download_url}
770         except Exception as e:
771             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
772
773     # --- CLI Debug Utility & Orchestration ---
774     def run_cli_diagnose_clip(video_path: str, timestamp: float):
775         """CLI debugging utility to examine segment properties around a timestamp."""
776         print("=" * 60)
777         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
778         print("=" * 60)
779
780         cfg = load_config()  # legacy wrapper still works, returns dict for now
781
782         # 1. Validation check diagnostics
783         print("[DIAGNOSTIC] Running validation framework...")
784         results = registry.run_all(video_path, cfg)
```

33. assistant (2026-06-22T09:09:09.486Z)

Perfect! Now I have enough context. Let me create the structured output with the complete mapping:

34. user (2026-06-22T09:09:49.347Z)

Structured output provided successfully

35. assistant (2026-06-22T09:10:18.680Z)

Perfect. Now let me provide a final summary as raw structured data for the workflow:

SUMMARY: Reel-Production Path & Nightly Regression Signal

I have successfully mapped the **end-to-end reel-production pipeline** and identified the stable regression signal.

REEL PRODUCTION PIPELINE

```
VIDEO INPUT
    ?
[/api/process or CLI --video-path]
    ?
_execute_processing (backend/main.py:439)
    ?? Hash video ? video_id = MD5(local_path)
    ?? Validate (backend/pipeline/validators)
    ?? Segmenter.process_video (backend/pipeline/segmenters/*)
         ?? Detects rallies ? [(start1, end1), (start2, end2), ...]
              ?? INSERT each into DB: play_segments
                   (id, video_id, start_time, end_time, duration, metadata)
    ?
_finalize_reingest (main.py:536)
    ?? prune_segments: keep new rows if successful
    ?
[/api/compile request with segment_ids list]
    ?
compile_highlights (backend/main.py:705)
    ?? Query play_segments by segment_ids
    ?? Filter by stitching.min_shot_count (from metadata['shot_count'])
    ?? Extract [(start, end), ...] time_pairs
    ?? VideoCompiler.compile_highlights
```

```
         ?? For each segment:
         ?   ?? _trim_one_segment: ffmpeg -ss -to -vf (watermark)
         ?        ? outputs clip_{N}.mp4
         ?? ffmpeg concat demuxer
              ? ONE stitched output .mp4 file
                 (output_dir/{output_filename}.mp4)
```

COUNTABLE, STABLE REGRESSION SIGNAL

Primary Signal: `SELECT COUNT(*) FROM play_segments WHERE video_id = ?`

This count is:
- **Stable:** Depends only on video content + segmenter configuration (not stitching params)
- **Deterministic:** Same video + same segmenter config ? same count (no stochasticity)
- **Queryable headlessly:** Direct SQL query on the SQLite DB (no UI required)
- **Early in pipeline:** Catches segmentation regressions before compilation

EXACT HEADLESS COMMANDS

1. Run segmentation (CLI single-shot, no server):
```bash
indexer --video-path /absolute/path/to/golden_video.mp4 \
        --segmenter motion \
        --output-dir output/nightly_e2e
```

2. Query segment count:
```python
from backend.infrastructure.database import Database
from backend.utils.hashing import compute_video_id

db = Database('output/nightly_e2e/rally_indexer.db')
video_id = compute_video_id('/absolute/path/to/golden_video.mp4')

with db._connect() as conn:
    count = conn.cursor().execute(
        'SELECT COUNT(*) FROM play_segments WHERE video_id = ?',
        (video_id,)
    ).fetchone()[0]

print(f"Segment count: {count}")
```

3. Assert against baseline:
```python
baseline = {"video1_md5": 3, "video2_md5": 5, "video3_md5": 4}
actual = {video_id: count}
assert actual[video_id] == baseline[video_id], f"Regression: {actual[video_id]} !=
{baseline[video_id]}"
```

CONFIG FOR GOLDEN VIDEOS

- **Segmenter:** `motion` (offline, reproducible, no API key/GPU needed)
- **Padding:** Default `begin_padding=0.5s, end_padding=0.5s` (configurable via `stitching`)
- **Input:** Local paths only (no cloud URIs)
- **Output DB:** `output/nightly_e2e/rally_indexer.db` (auto-created)

BASELINE CHECKPOINT

Create and commit: `eval_baselines/nightly_e2e_baselines.json`

```json
```

```
{
  "timestamp": "2026-06-22T00:00:00Z",
  "git_commit": "<HEAD commit hash>",
  "segmenter": "motion",
  "videos": [
    {
      "name": "golden_video_1.mp4",
      "video_id": "<MD5 hash>",
      "expected_play_segments_count": 3
    },
    {
      "name": "golden_video_2.mp4",
      "video_id": "<MD5 hash>",
      "expected_play_segments_count": 5
    },
    {
      "name": "golden_video_3.mp4",
      "video_id": "<MD5 hash>",
      "expected_play_segments_count": 4
    }
  ],
  "_doc": "Snapshot of play_segments row counts from golden video set at a known-good HEAD.
Nightly regression tripwire."
}
```

DATA FLOW FOR /api/compile

```
Client sends: CompileRequest(video_id, segment_ids=[1,2,3], output_filename="reel.mp4")
  ?
query_play_segments(video_id, {}) ? ALL segments for video
  ?
Filter by segment_ids AND stitching.min_shot_count
  ?
Extract time_pairs: [(s["start_time"], s["end_time"]), ...]
  ?
VideoCompiler.compile_highlights(video_path, time_pairs, output_filename)
  ?? Merge overlapping ranges
  ?? Trim each with padding + watermark (per-clip re-encode)
  ?? ffmpeg concat ? ONE .mp4 at output_dir/reel.mp4
  ?
Response: {"status": "success", "filepath": "/abs/path/to/reel.mp4",
           "download_url": "/api/highlights/<video_id>/reel.mp4"}
```

KEY FILE REFERENCES

- **Entry point:** `backend/main.py::_execute_processing` (line 439) and `compile_highlights` (line 705)
- **Database:** `backend/infrastructure/database.py::Database` (line 24)
- **Segmentation result persistence:** `database.py::add_play_segment` (line 459)
- **Stitching:** `backend/pipeline/stitcher/compiler.py::VideoCompiler` (line 18) and `compile_highlights` (line 305)
- **Config:** `backend/config/models.py::StitchingConfig` (line 255): `begin_padding`, `end_padding`, `min_shot_count`
- **Hashing:** `backend/utils/hashing.py::compute_video_id`